
Unit Testing Using PyUnit

An Introduction

What is PyUnit?

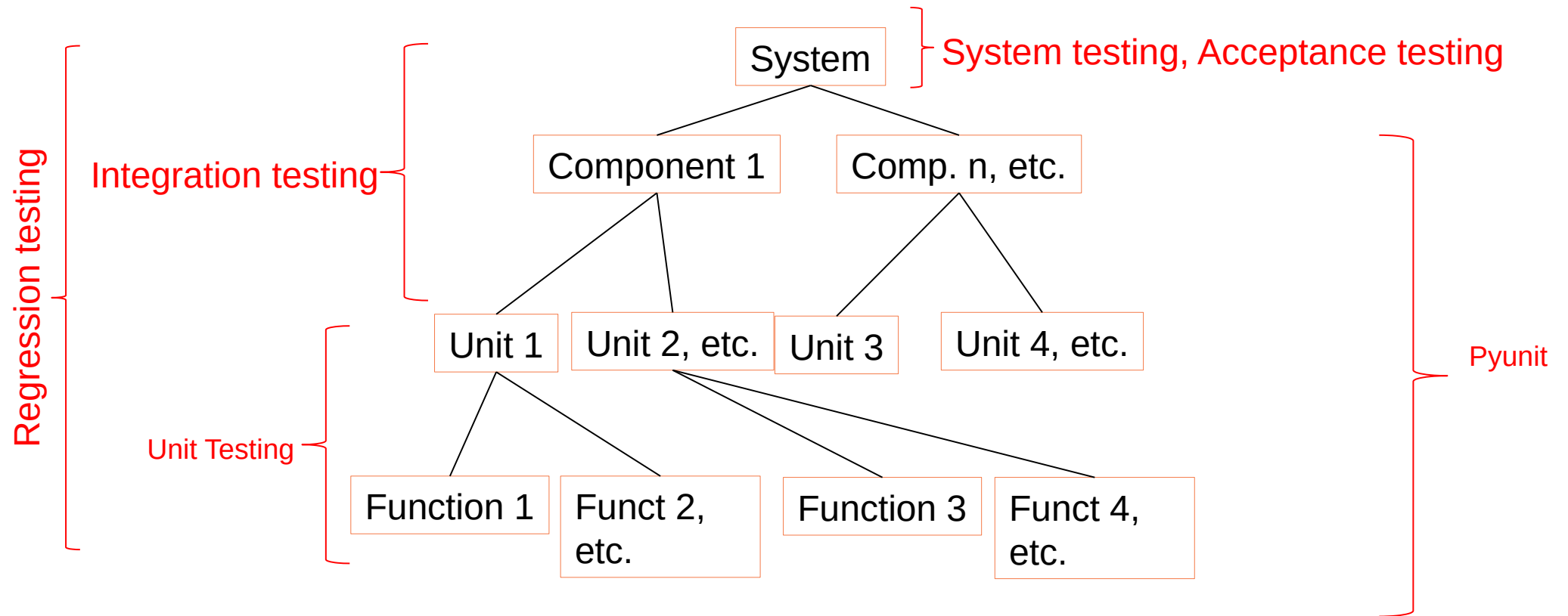
- PyUnit is a unit testing framework for Python
- PyUnit is a Python language version of JUnit
- Plays a crucial role in test-driven development
- Advocates the idea of "first testing then coding" which is aligned with a test driven development (TDD) philosophy
 - Set up the test data for a piece of code
 - Test the test set up
 - Then write the code
 - Then unit test the code using the test set up
- Unit testing can increase programme productivity
 - If implemented correctly, less time will be required for debugging / resolving defects

Why use unit tests?

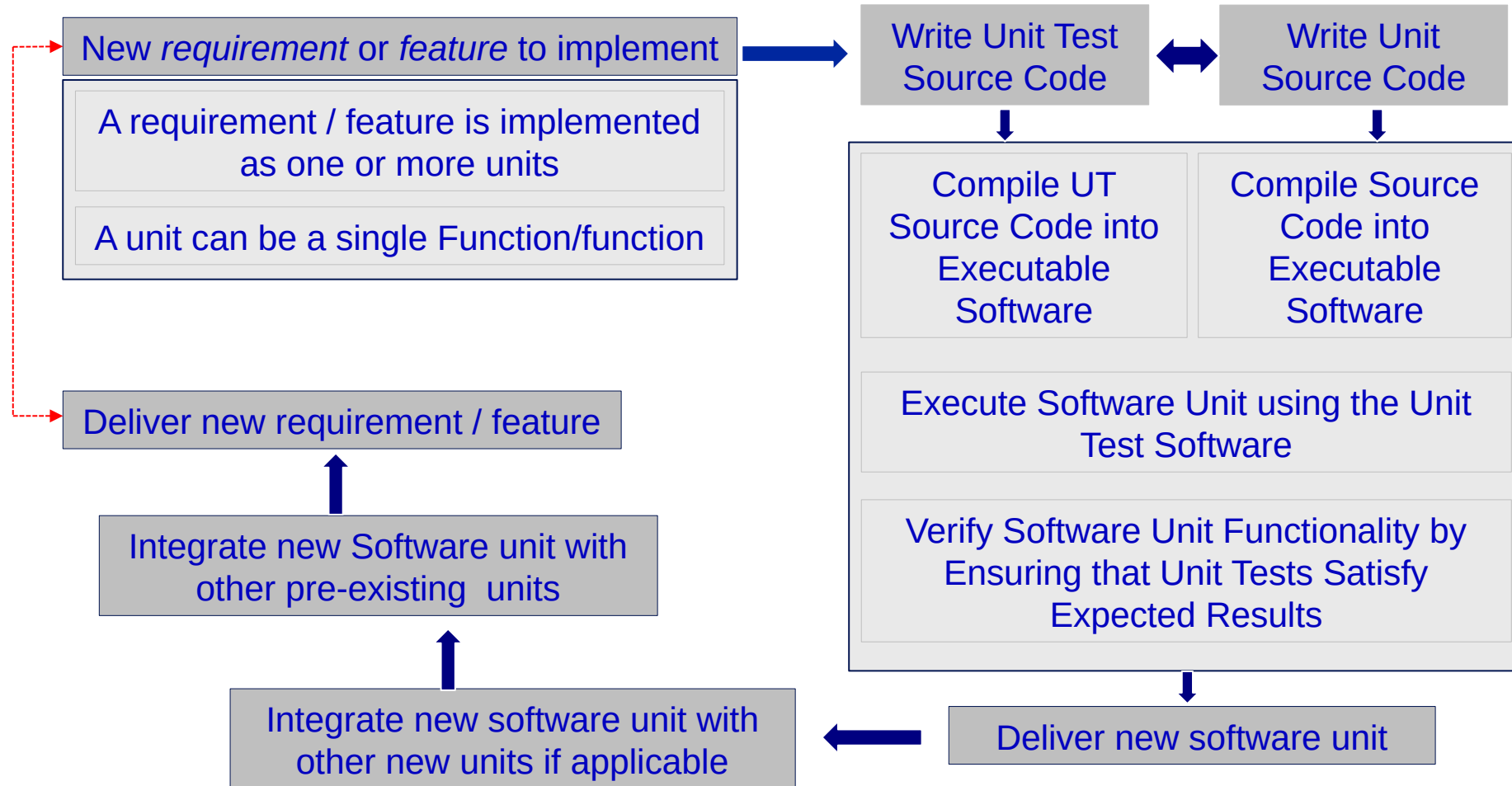
- Unit tests help developers to verify that the logic / behaviour of a piece of code/software is correct.
- Re-performing unit tests (automatically) can identify software regressions introduced by changes in the code.
- Unit tests are therefore very useful for regression testing
- Unit testing frameworks can be harnessed for integration and system level testing also.
- May not be sensible to write unit tests for all code.

Where unit testing fits in...

Software System Decomposition



Sequencing of unit testing - generalised



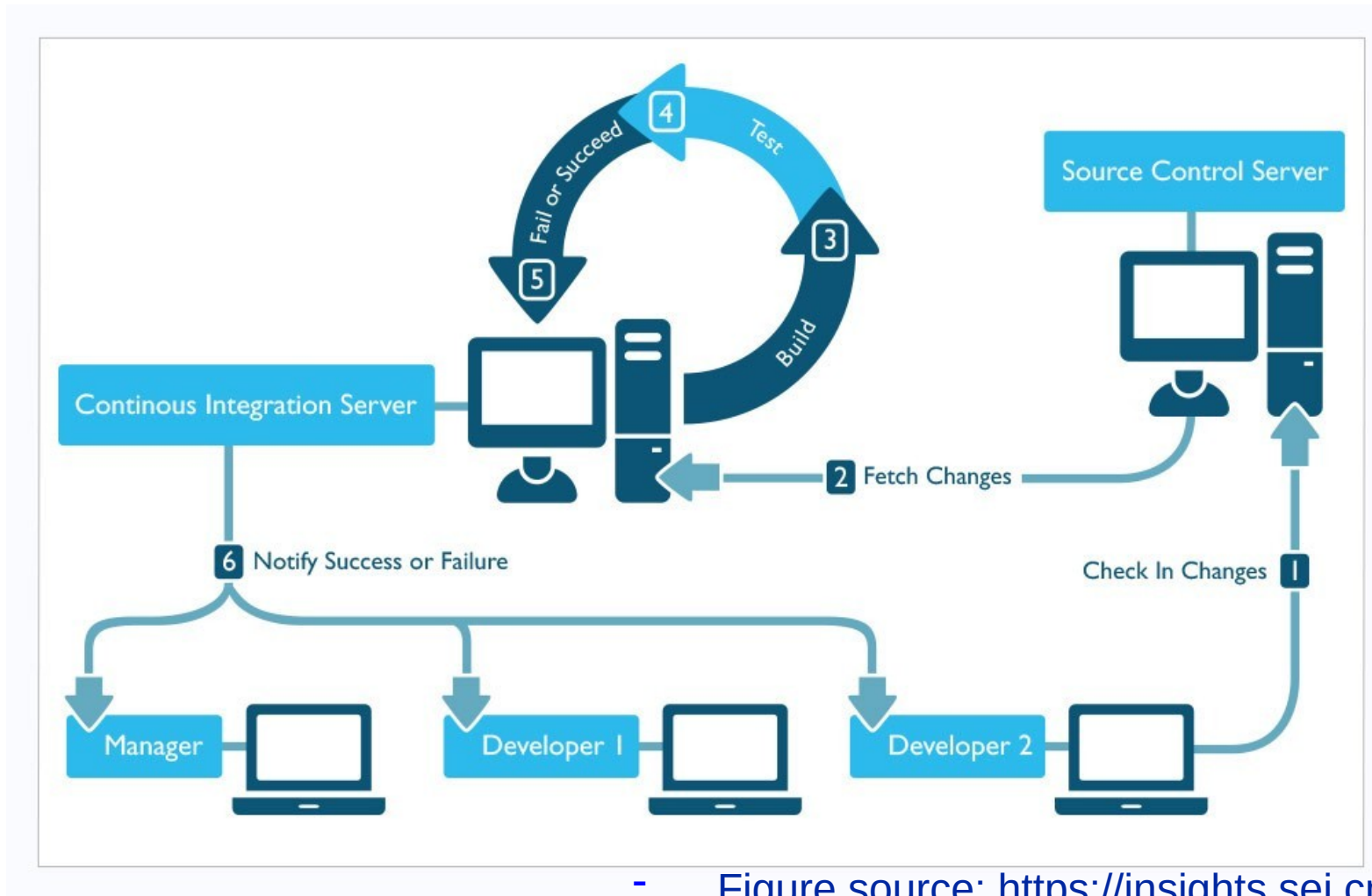
What happens when unit tests fail?

- For an individual programmer:
 - Their compilation will fail until the unit test passes
- For a group programming together:
 - The group build/compile will fail unless all unit tests pass
 - Manifestly more embarrassing for the individual who breaks the build!
- Major implications for Continuous Integration systems that are a feature of most modern software development

Continuous Integration (CI) systems

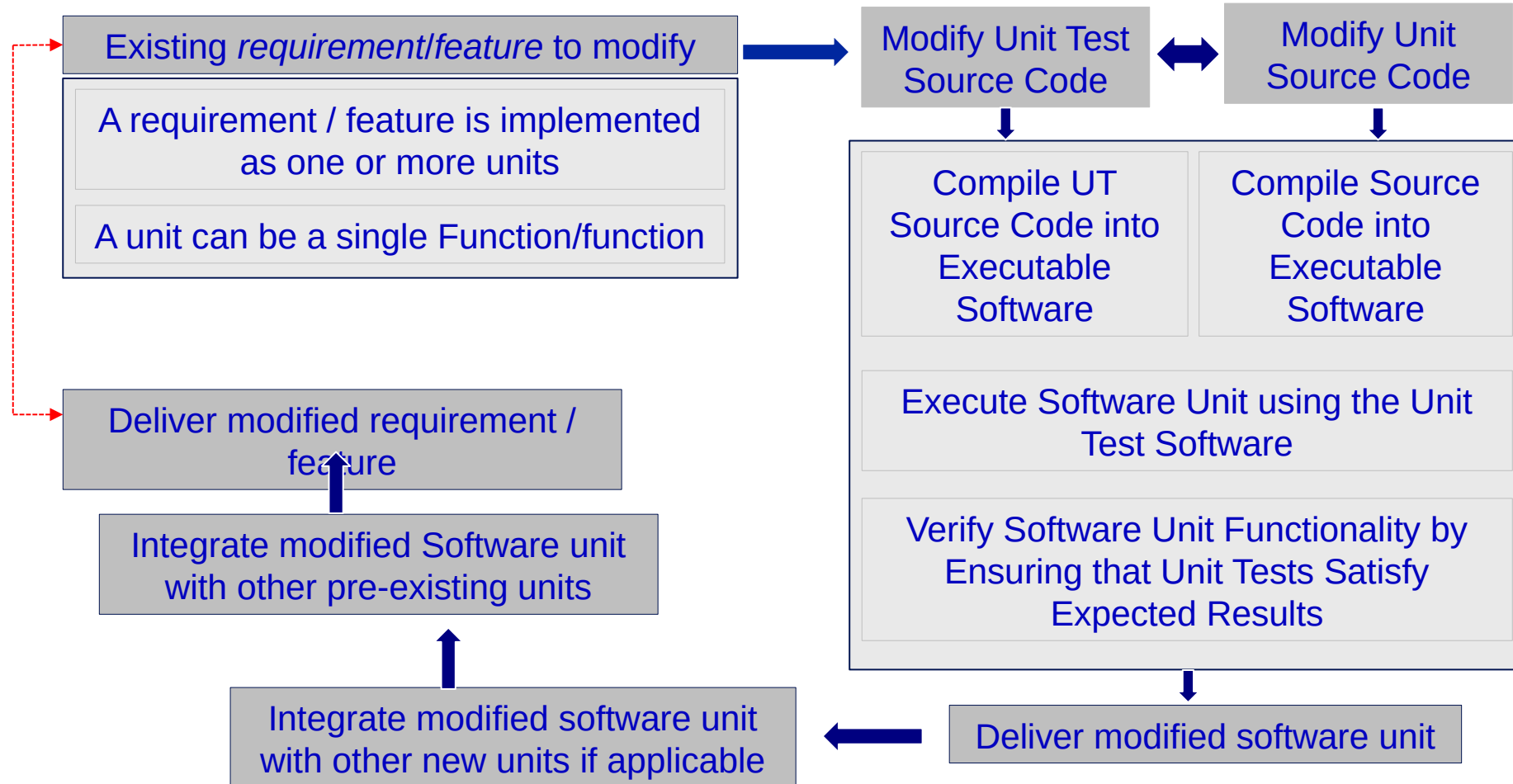
- Continuous Integration (CI) is a development practice that requires developers to integrate code into a shared repository several times a day.
- Each check-in is then verified by an automated build, allowing teams to detect problems early.
- By integrating regularly, you can detect errors quickly, and locate them more easily.
- Automated builds are inherently associated with unit testing.

Continuous Integration (CI) systems



- Figure source: <https://insights.sei.cmu.edu>

Sequencing of unit testing (2)



What is PyUnit?

- PyUnit is an open source framework
- Used for writing and running unit tests (and test suites)
- Part of the python standard library (*import unittest*)
- Provides assertions for testing expected results
 - E.g. *assertEqual()* to check if an actual value corresponds to the expected value
- Provides test runners for executing tests
 - E.g. a Function called *TestRunner* that can invoke, execute and report on unit test case results (can be used to run suites of tests)

PyUnit – automation and regression

- PyUnit runs automatically (once the tests are written!)
- PyUnit checks results automatically – and flags any errors
- No errors means that all the unit tests have passed as expected
- PyUnit tests can be organised into suites of test cases
- Test suites can be executed any time and much more quickly than manual testing
- This has positive impacts for regression testing and associated software release pace.

What is a PyUnit test case?

- A piece of code that checks (ensures?) that another piece of code (e.g. a function / method) works as expected
- PyUnit is a testing framework that enables us to quickly develop, execute and re-execute unit test cases for python code

What is a PyUnit test case?

- A unit test case is characterised by
 - a known input, and
 - an expected output.
- Both inputs and expected outputs are identified **before** the test is executed.

How many unit test cases?

- It is suggested that there should be at least two unit test cases for each requirement
 - one positive test, and
 - one negative test.
- Where a requirement has sub-requirements, each sub-requirement should also have at least two test cases: one positive and one negative.

What are positive / negative test cases? (1)

- No commonly accepted general definition for these terms!
- Positive test cases – test the behaviour of the code using data that is expect to be received in a correct operational mode
 - E.G. The year entered is valid, i.e. “2017”
- Negative test cases are – test the behaviour of the code when incorrect data is entered (i.e. how does the code handle instances of incorrect data)
 - E.G. The year entered is invalid, i.e. “1822”, or “2050”, or “asde”, or “-dfg”
- It is important to adopt both perspectives.

What are positive / negative test cases? (2)

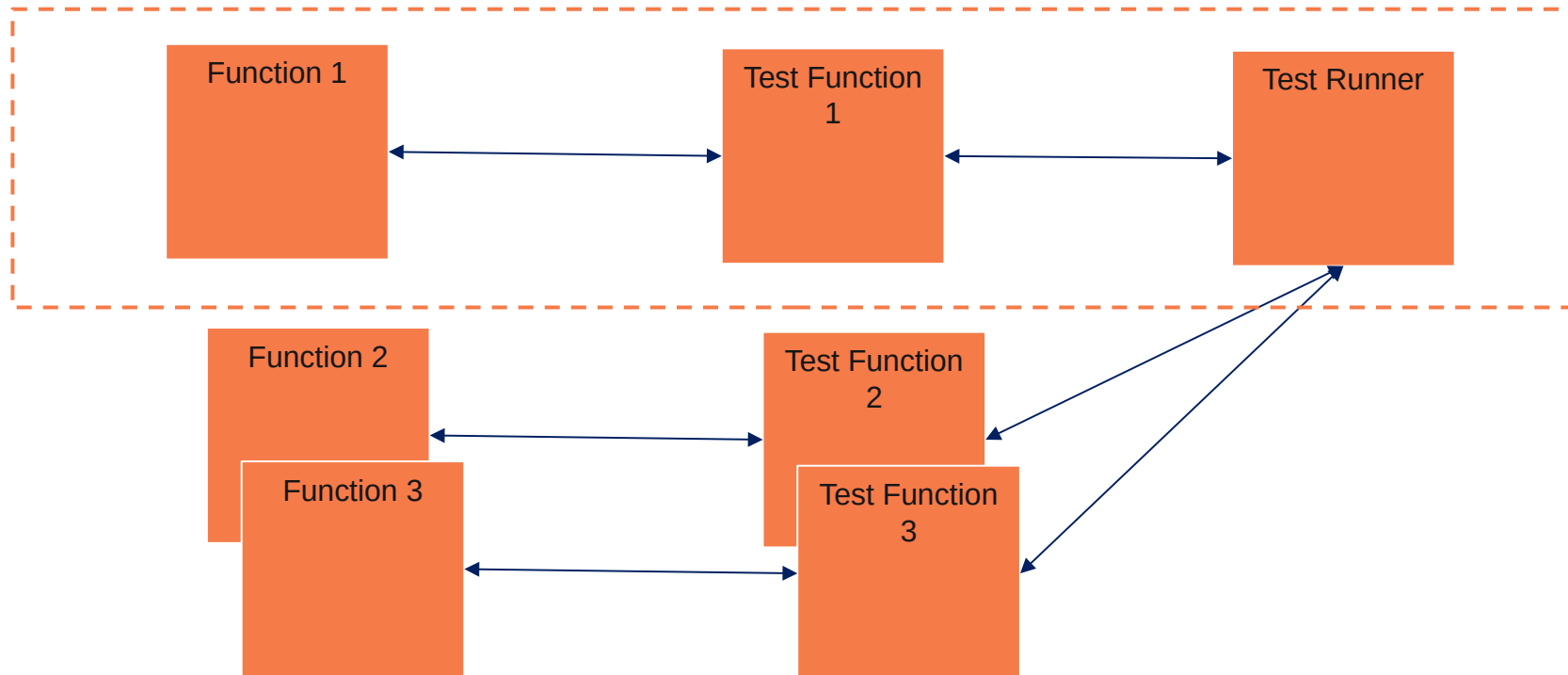
- An alternative definition:
 - Positive testing is the testing for something that should happen, does happen.
 - Negative testing is the testing for something that should not happen, does not happen
- Whatever definition is adopted, a close examination of the requirement / user story / feature is necessary to design both positive and negative test cases.
- When completed, all tests should be complete and they should all pass

Unit test tips

- Provide meaningful / accurate messages in assert statements:
 - It may be (more) obvious what the issue is, meaning that it will be easier to identify and fix.
 - It won't always be the original developer (or any developer) who will notice/manage reported unit test failures
- But only add messages to the assert statement if they represent an improvement from the standard error message that will be issued in the case of a unit test failure.

3 different .py files...

- A *TestSuite* is used by convention for *running* suites of tests.



PyUnit Assertions

- There are many ways to use PyUnit assertions
- A list of PyUnit assertion usage is available here:
<https://docs.python.org/2/library/unittest.html#assert-methods>
- *assertEqual()* and *assertNotEqual()* unit test assert methods are perhaps the two most basic / well known

PyUnit Assertions

Method

`assertEqual(a, b)`

`assertNotEqual(a, b)`

`assertTrue(x)`

`assertFalse(x)`

`assertAlmostEqual(a, b)`

`assertNotAlmostEqual(a, b)`

Checks that

`a == b`

`a != b`

`bool(x)` is True

`bool(x)` is False

a is considered equal to b in view of desired precision

a is not considered equal to b in view of desired precision

Where to locate unit tests?

- No universally agreed response to this question.
- From a code management perspective, there are some benefits to separating the unit test code from the actual code – but the location of the unit test code is a subject of some contention.
- Some advocate that the unit tests should reside alongside the code, perhaps within a sub directory.
- Others advocate that the unit tests should be elsewhere in the directory hierarchy, separated from the code.

Why the location debate?

- Some advocate shipping unit test code to production systems (and in some cases this is desirable)
- Others advocate that unit test code should never be shipped to production systems.
- And there is some debate in terms of sequencing, with some advocating that unit tests should only be executed once there are no underlying build issues (others advocate that this is not necessary or desirable). Isn't a basic compile error (e.g. arising from a syntax error) different to a unit test failure?
- So what are the benefits of these different approaches?